Trabajo Práctico Integrador

Arquitectura de CI Centralizada y Seguridad de Contenedores

> **📖 El Escenario:**
>
> Fuiste contratado como Lead DevOps Engineer en una startup tecnológica. Actualmente, la empresa tiene 50 microservicios y los desarrolladores están copiando y pegando el mismo archivo `.github/workflows/main.yml` en todos los repositorios. Esto generó un caos: tiempos de build larguísimos, imágenes de Docker con vulnerabilidades críticas en los registros y alta exposición a errores manuales.

> **🎯 Tu Misión:**
>
> Diseñar e implementar una estrategia de Integración Continua (CI) centralizada que obligue a todos los microservicios a pasar por un estándar estricto de optimización, análisis de código y seguridad antes de publicar una imagen al registro de contenedores.

📦 Entregables Requeridos

Para aprobar esta actividad, deberás entregar los enlaces a **dos repositorios públicos** en tu cuenta de GitHub, los cuales deben interactuar entre sí:

- **Repositorio 1:** `ci-central`. Este repositorio alojará tus plantillas corporativas y acciones compuestas.
- **Repositorio 2:** `api-demo`. Un repositorio simulando un microservicio (ej. aplicación básica en Node.js o Python) que consuma tu plantilla central de CI.

🛠️ Fases de Implementación

Fase 1: El Repositorio Central (ci-central)

En este repositorio deberás construir las herramientas reutilizables que orquestarán los despliegues de la empresa.

1. **Composite Action (Preparación del Entorno):**
    - Crea un archivo `action.yml` dentro de un directorio dedicado (ej. `setup-ambiente/`).
    - Esta acción debe descargar el código fuente (checkout) y configurar el lenguaje de programación elegido.
    - *Requisito de Optimización:* Debe implementar obligatoriamente el **caché de dependencias** (ej. caché de `npm` o `pip`) para acelerar ejecuciones futuras.
2. **Workflow Reutilizable (El corazón del sistema):**
    - Crea un workflow utilizando el disparador `workflow_call`.
    - Debe recibir dinámicamente el nombre de la imagen como *input*, y contar con los permisos/secretos necesarios para escribir en el registro de paquetes.
    - **Paso A:** Llamar a tu Composite Action creada en el punto anterior.
    - **Paso B (Seguridad):** Construir la imagen de Docker localmente (`load: true`) y ejecutar **Trivy**. Debe abortar el pipeline (`exit-code: 1`) si detecta vulnerabilidades de nivel `CRITICAL`.
    - **Paso C (Publicación):** Si el escaneo es exitoso, generar etiquetas dinámicas (usando el Git SHA) y publicar la imagen final en **GitHub Container Registry (GHCR)**.

Fase 2: El Repositorio Consumidor (api-demo)

Este repositorio simulará ser uno de los 50 microservicios de la empresa y debe ser lo más limpio posible.

1. **Código y Dockerfile:**
    - Debe contener el código fuente de tu API y un `Dockerfile` optimizado.
    - El Dockerfile debe implementar **Multi-stage build**, separando estrictamente la fase de compilación de la fase de ejecución productiva.
2. **El Pipeline del Desarrollador:**
    - Crea el archivo de workflow principal.
    - *Requisito de Optimización:* Implementar una regla de **Concurrencia** para cancelar ejecuciones redundantes en la misma rama si hay pushes consecutivos rápidos.
    - *Consumo de la Plantilla:* Este archivo **no debe contener pasos de construcción**. Únicamente debe llamar a tu plantilla central (`uses: tu-usuario/ci-central/.github/workflows/...`).

Fase 3: Demostración del Bloqueo (Gatekeeper)

Para auditar la eficacia de las medidas de seguridad inyectadas:

- Modifica temporalmente el `Dockerfile` de tu `api-demo` para basarse en una imagen extremadamente antigua (ej. `node:14-alpine`).
- Realiza un push a tu rama principal.
- El pipeline **debe fallar** de forma predecible en el paso de escaneo (Trivy), previniendo la publicación en el registro. Deja esta ejecución fallida visible en el historial de Actions para que pueda ser evaluada.

🏆 Rúbrica de Evaluación

Criterio	Detalle de evaluación
Arquitectura DRY	El repositorio del microservicio está limpio de lógica CI. Toda la orquestación (Docker, Trivy, GHCR) reside centralizada. Se observa un correcto paso de contextos e `inputs` entre repositorios.
Optimización	El caché de dependencias (`actions/cache`) funciona correctamente dentro de la Composite Action. La concurrencia interrumpe los pipelines encolados obsoletos.
Seguridad Inmutable	Trivy frena el build ante CVEs críticos. El archivo Dockerfile utiliza Multi-stage builds descartando las herramientas de desarrollo en la imagen productiva.
Gestión de Artefactos	Las imágenes publicadas en GHCR contienen metadatos dinámicos vinculados al hash del commit exacto de Git (evitando la dependencia exclusiva del tag `latest`).

🚀 Bonus Track (Opcional)

Implementa **GitHub Environments** en tu repositorio `api-demo`. Añade un Job final llamado `despliegue-simulado` que requiera obligatoriamente una **Aprobación Manual** por parte de un revisor antes de imprimir por consola "Iniciando despliegue hacia Producción".